



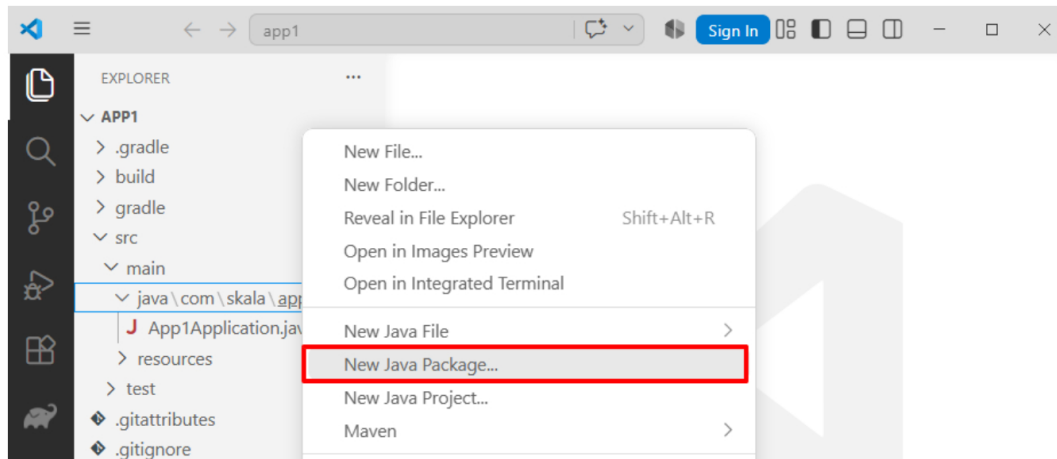
Do it! 자바 완전 정복

<https://www.youtube.com/watch?v=jALoE5BGF1s&list=PLG7te9eYUi7toebNnbA1cZNRDoUcHmsGd>
Chapter 10 부터 13 까지 꼭 보세요.

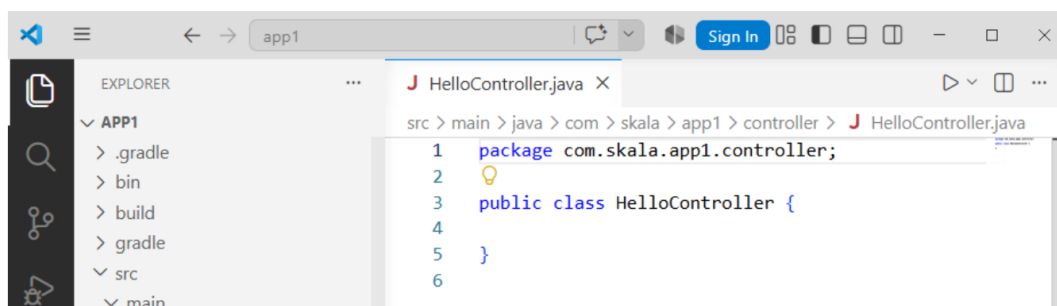
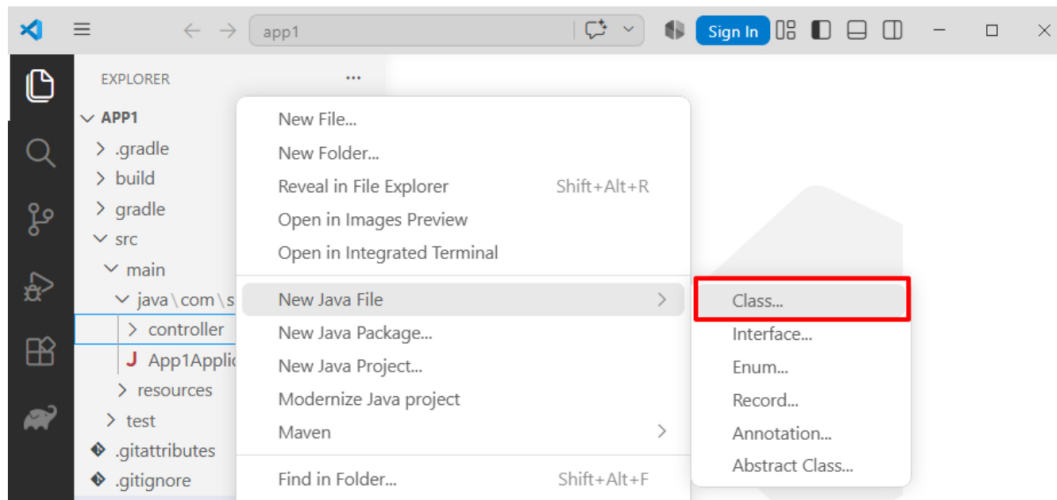
REST API 추가 예

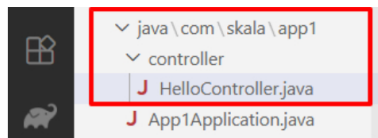
Controller 추가

controller 패키지 생성



class 생성



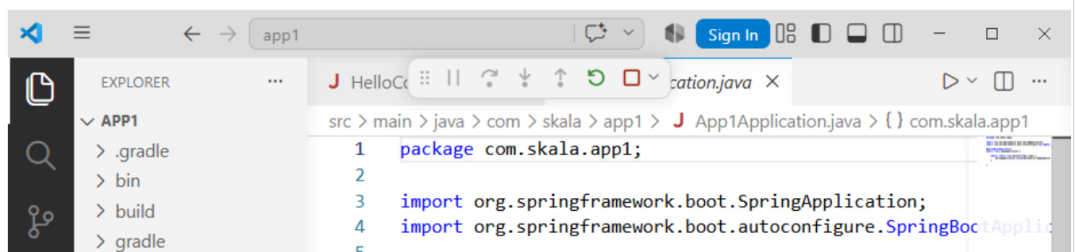
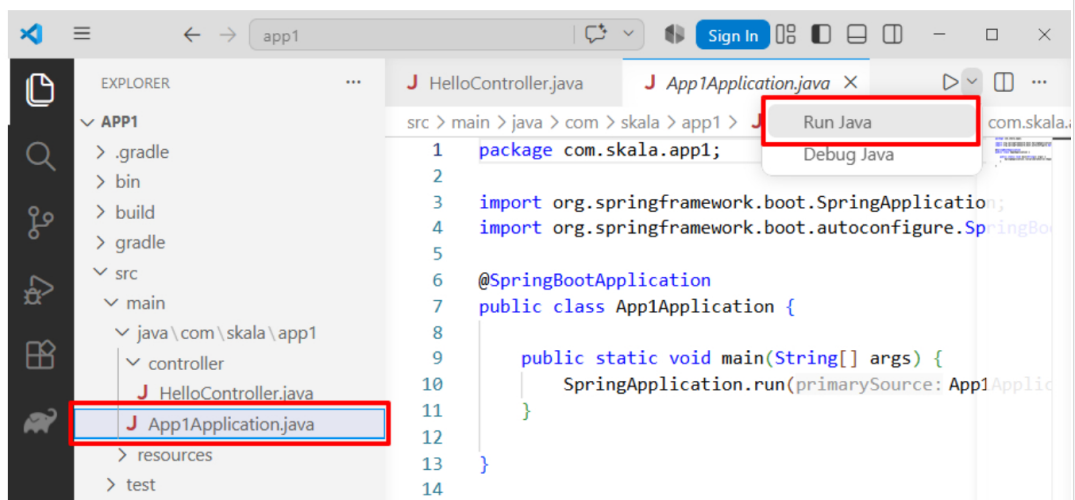


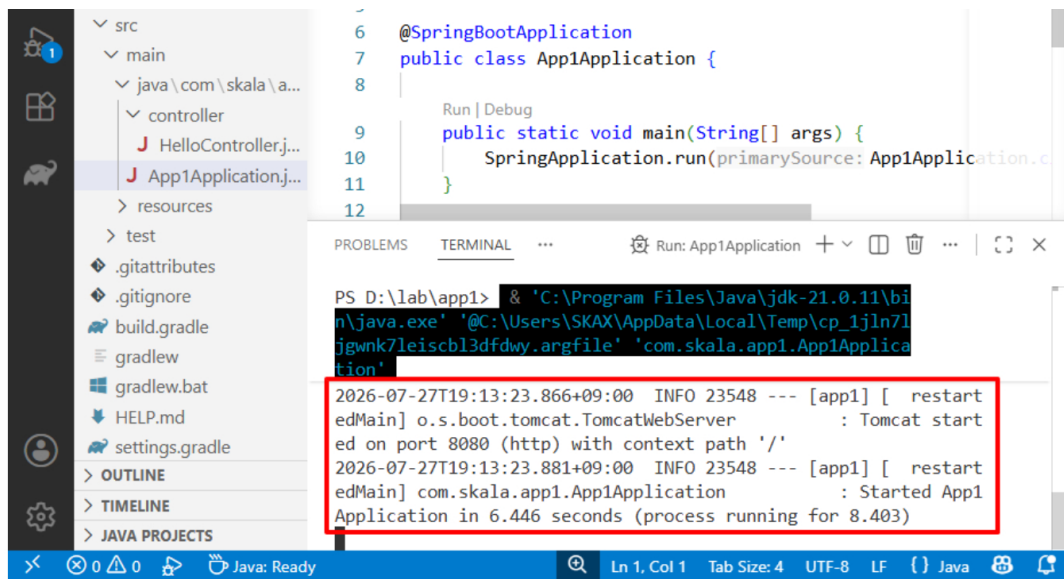
```
package com.skala.app1.controller;
```

```
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RequestMapping;  
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController  
@RequestMapping("/api")  
public class HelloController {  
    @GetMapping("")  
    public String home() {  
        return "home";  
    }  
  
    @GetMapping("/hello")  
    public String hello() {  
        return "hello, world!!!";  
    }  
}
```

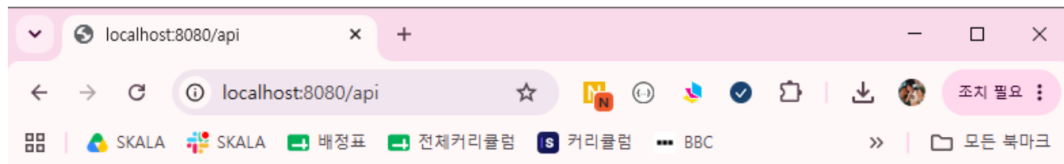
애플리케이션 실행



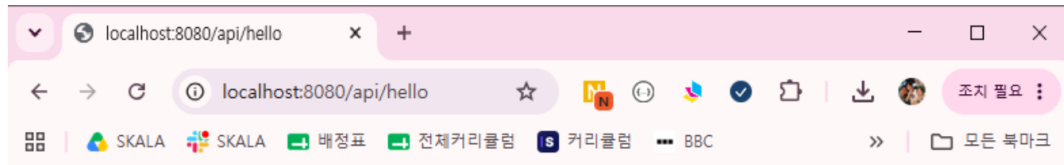


브라우저를 이용해서 동작을 확인

http://localhost:8080/api



http://localhost:8080/api/hello



Post 방식과 DELETE 방식의 요청을 처리하는 메서드를 추가

```
package com.skala.app1.controller;
```

```
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
@RequestMapping("/api")
public class HelloController {
    @GetMapping("")
    public String home() {
        return "home";
    }
}
```

```
@GetMapping("/hello")
public String hello() {
```

```

        return "hello, world!!!";
    }

    @PostMapping("/post")
    public String post() {
        return "request by POST";
    }

```

```

    @DeleteMapping("/delete")
    public String delete() {
        return "request by DELETE";
    }
}

```

GET 방식 이 외의 요청을 테스트하기 위해서는 REST API 테스트 도구가 필요

대표적인 도구로 Postman이 있으며, CLI 기반의 도구로 curl이 있음

```

C:\Users\SKAX> curl -X GET http://localhost:8080/api/hello
hello, world!!!

```

```

C:\Users\SKAX> curl -X DELETE http://localhost:8080/api/delete
request by DELETE

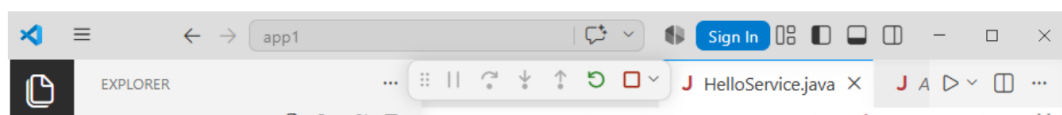
```

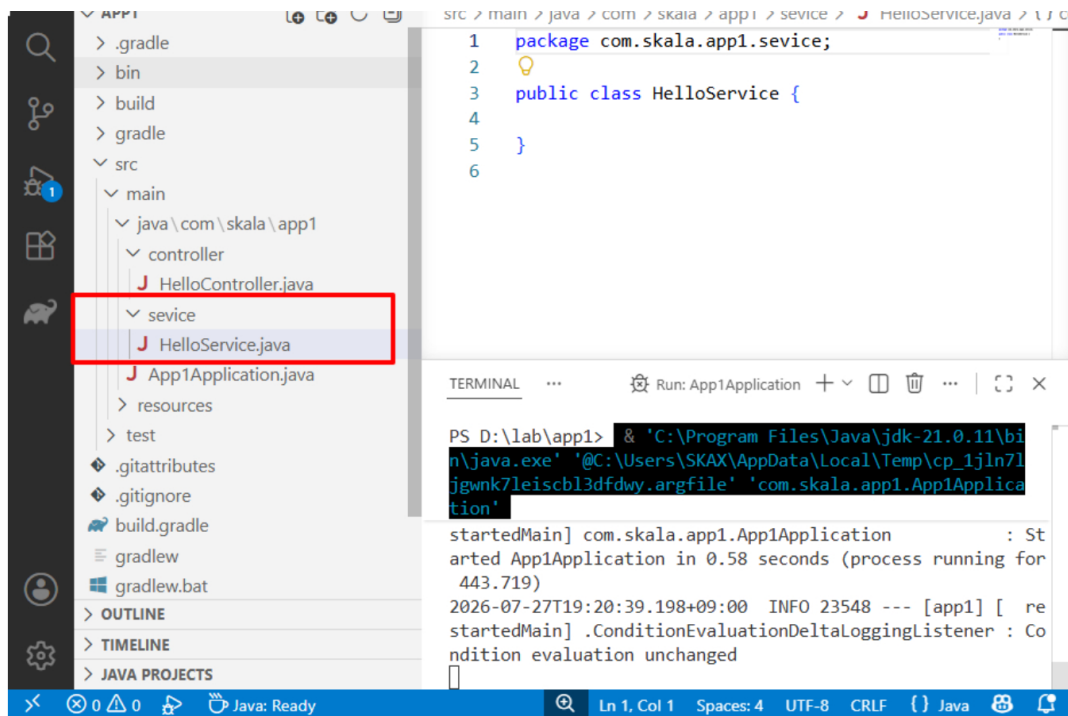
```

C:\Users\SKAX> curl -X DELETE http://localhost:8080/api/delete -v
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:8080...
* Connected to localhost (::1) port 8080
* using HTTP/1.x
> DELETE /api/delete HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.13.0
> Accept: */*
>
< HTTP/1.1 200
< Content-Type: text/plain;charset=UTF-8
< Content-Length: 17
< Date: Mon, 27 Jul 2026 10:19:15 GMT
<
request by DELETE* Connection #0 to host localhost left intact

```

서비스 추가





```
package com.skala.app1.service;

import org.springframework.stereotype.Service;

@Service
public class HelloService {
    public String hello() {
        return "Say hello from HelloService";
    }
}
```

HelloController 수정 ← HelloService를 이용하도록 수정

```
package com.skala.app1.controller;

import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
```

```
import org.springframework.web.bind.annotation.RestController;

import com.skala.app1.service>HelloService;

@RestController
@RequestMapping("/api")
public class HelloController {
    // 의존 객체를 필드로 정의
    private final HelloService helloService;

    // 생성자를 이용한 의존 주입
    public HelloController(HelloService helloService) {
        this.helloService = helloService;
    }

    @GetMapping("/")
    public String home() {
        return "home";
    }

    @GetMapping("/hello")
    public String hello() {
        // 의존 객체의 기능을 사용
    }
}
```

```

        return helloService.hello();
    }

    @PostMapping("/post")
    public String post() {
        return "request by POST";
    }

    @DeleteMapping("/delete")
    public String delete() {
        return "request by DELETE";
    }
}

```

실행

C:\Users\SKAX> curl -X GET http://localhost:8080/api/hello

Say hello from HelloService

⇐ HelloService의 hello() 메서드가 반환한 값이 출력됨

실습 프로젝트 구조

프로젝트 생성 ⇒ <https://start.spring.io/> 활용


spring initializr

Project

☒ Gradle - Groovy
 ☐ Gradle - Kotlin
 ☐ Maven

Language

☒ Java
 ☐ Kotlin
 ☐ Groovy

Spring Boot

☐ 4.1.1 (SNAPSHOT)
 ☒ 4.1.0
 ☐ 4.0.8 (SNAPSHOT)
 ☐ 4.0.7

Project Metadata

Group

com.example

Artifact

demo

Package name

com.example.demo

Packaging

☒ Jar
 ☐ War

Configuration

☒ Properties
 ☐ YAML

Java

☐ 26
 ☐ 25
 ☒ 21
 ☐ 17

Dependencies

ADD ... CTRL + B

Lombok

DEVELOPER TOOLS

Java annotation library which helps to reduce boilerplate code.

Spring Boot DevTools

DEVELOPER TOOLS

Provides fast application restarts, LiveReload, and configurations for enhanced development experience.

Spring Web

WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

GENERATE

CTRL + ↵

EXPLORE

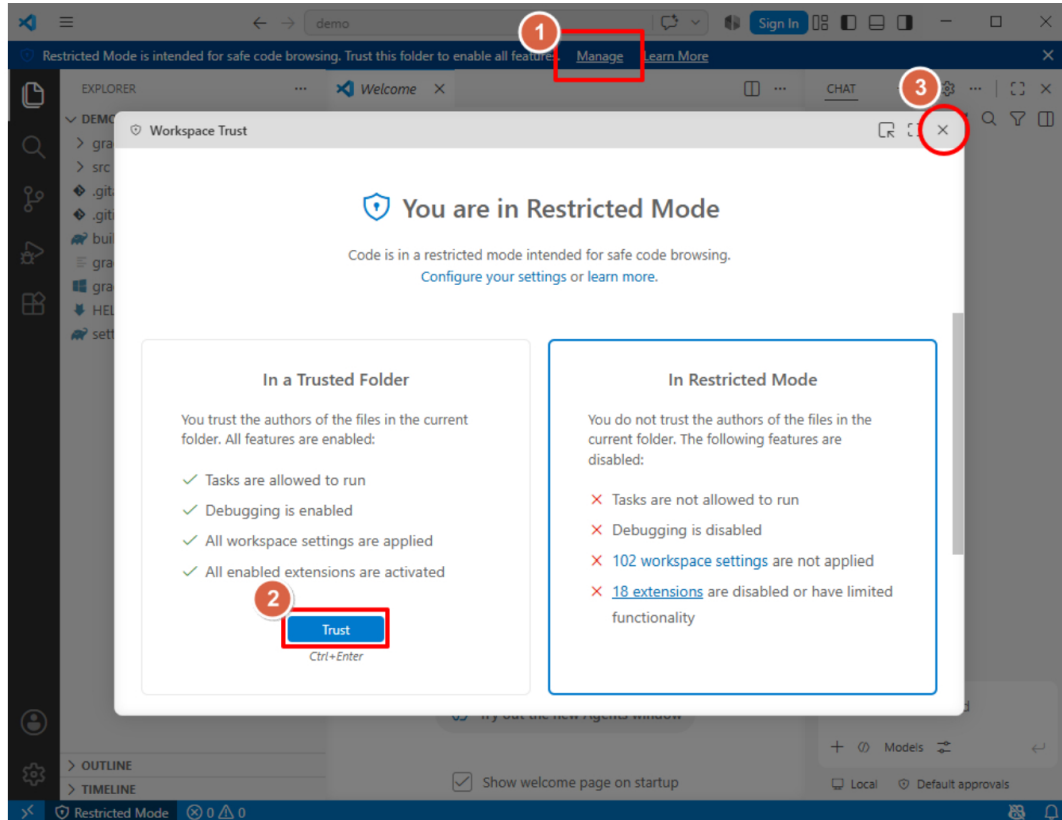
CTRL + SPACE

...

내려받은 demo.zip 파일을 원하는 위치에 압축 해제 ⇒ 압축 해제한 디렉터리가 프로젝트 디렉터리가 됩니다.

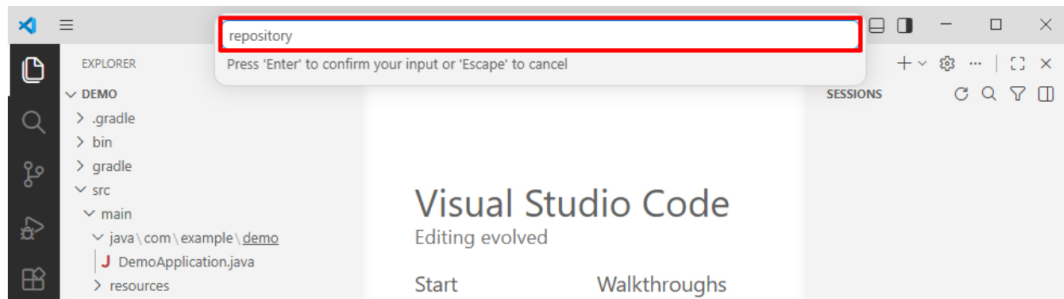
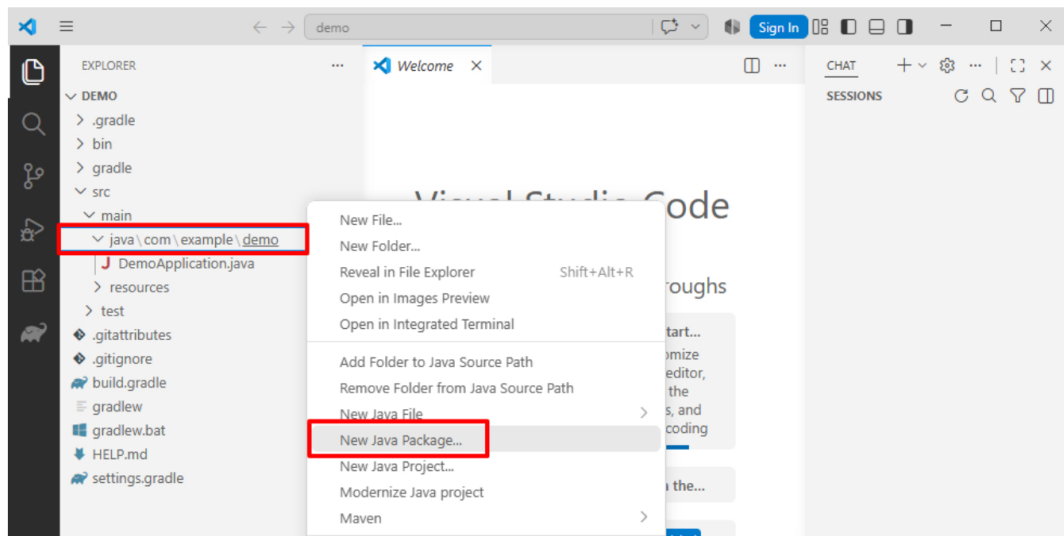
터미널을 실행해서 압축 해제한 디렉터리로 이동한 후 `code .` 명령어 실행 ⇐ 해당 디렉터를 프로젝트 루트로 설정한 상태로 VSCode를 실행

해당 디렉터를 신뢰할 수 있는 디렉터리로 등록

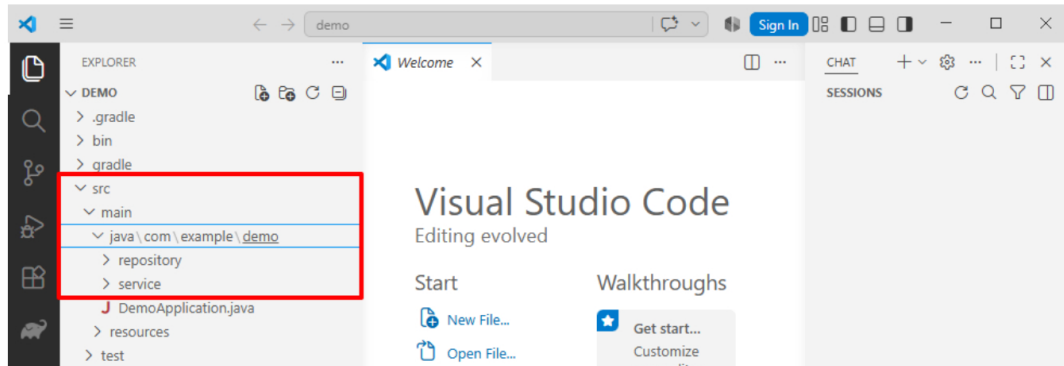


디렉터리 생성 (76 참조)

`src > main > java > com > example > demo` 디렉터리 아래에 repository 디렉터리 생성



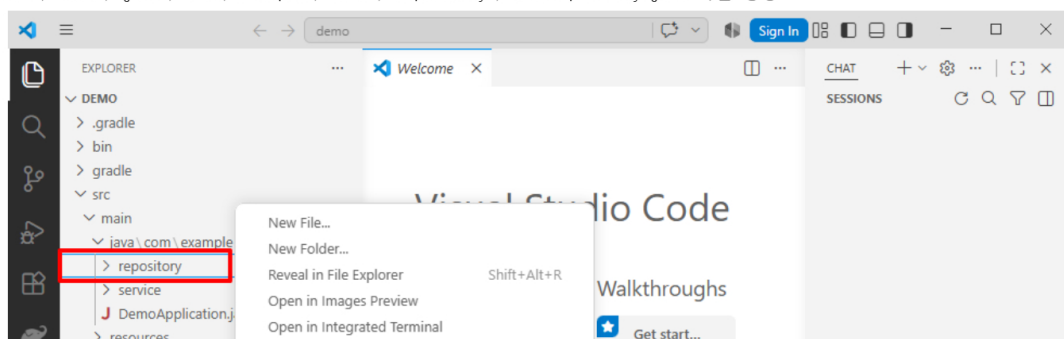
같은 방식으로 src > main > java > com > example > demo 디렉터리 아래에 service 디렉터리 생성

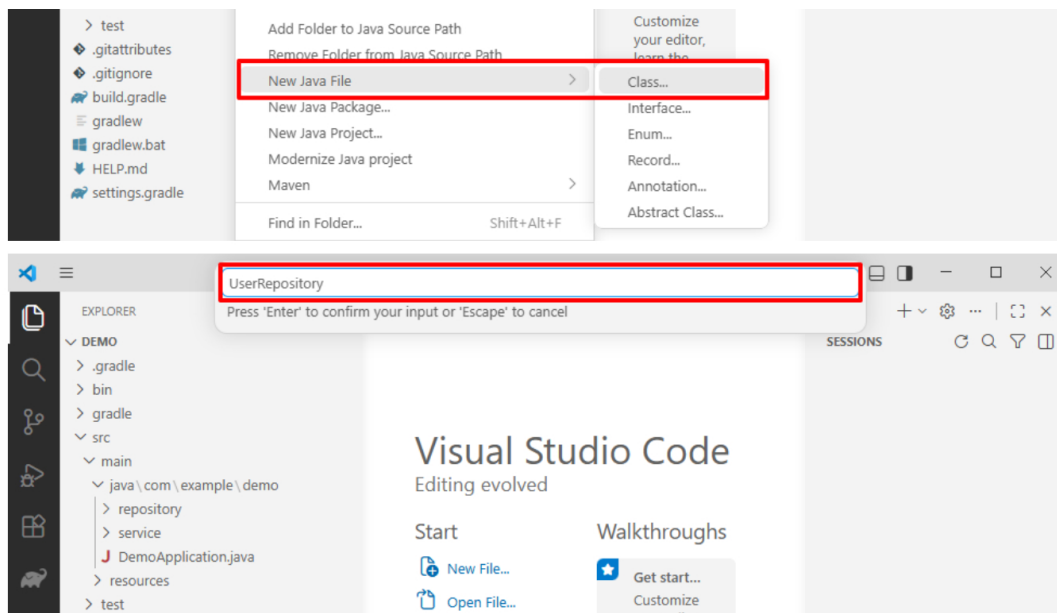


Java 파일 생성

UserRepository

src > main > java > com > example > demo > repository > UserRepository.java 파일 생성





```
package com.example.demo.repository;

import org.springframework.stereotype.Repository;

@Repository
public class UserRepository {
    public String findUser() {
        return "홍길동";
    }
}
```

UserService

같은 방식으로 src > main > java > com > example > demo > service > UserService.java 파일 생성



```
package com.example.demo.service;

import org.springframework.stereotype.Service;

import com.example.demo.repository.UserRepository;

@Service("userService")
public class UserService {
    // 의존 객체
    private UserRepository userRepository;

    // 생성자를 이용해서 의존 주의
    public UserService(UserRepository userRepository) {
```

```

        this.userRepository = userRepository;
    }

    // 서비스 메서드 <= 비즈니스 로직을 처리하는 메서드
    public void printUser() {
        System.out.println("사용자: " + userRepository.findUser());
    }
}

```

애플리케이션 클래스(DemoApplication)에 서비스

```
package com.example.demo;
```

```

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;

```

```

import com.example.demo.service.UserService;

@SpringBootApplication
public class DemoApplication {

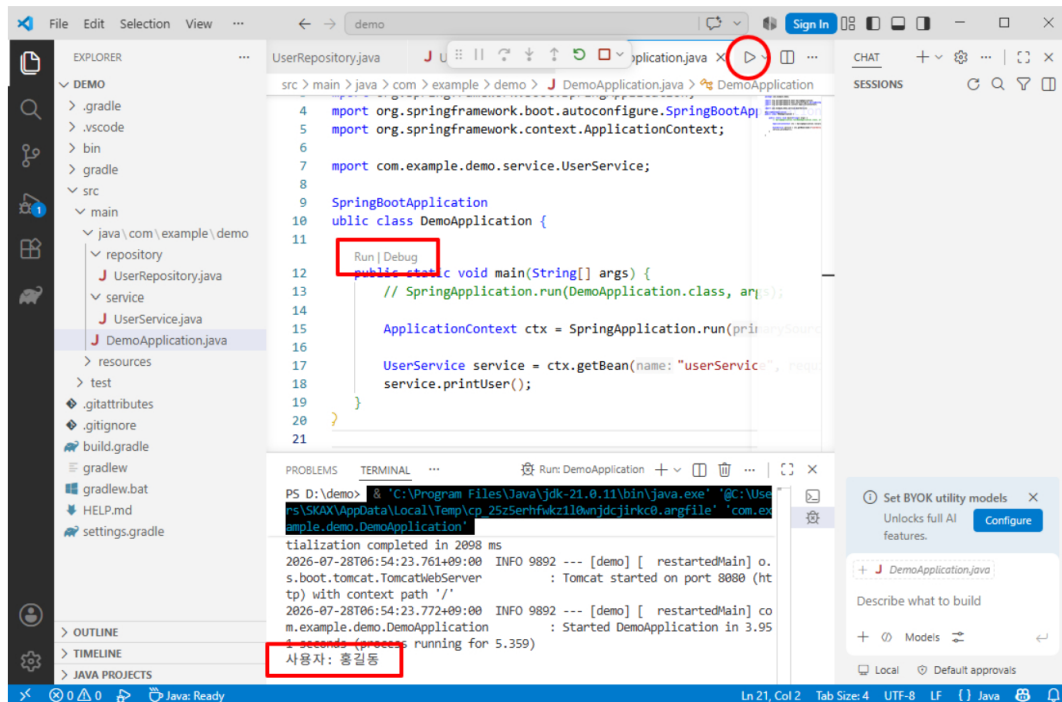
    public static void main(String[] args) {
        // SpringApplication.run(DemoApplication.class, args);

        ApplicationContext ctx = SpringApplication.run(DemoApplication.class, args);

        UserService service = ctx.getBean("userService", UserService.class);
        service.printUser();
    }
}

```

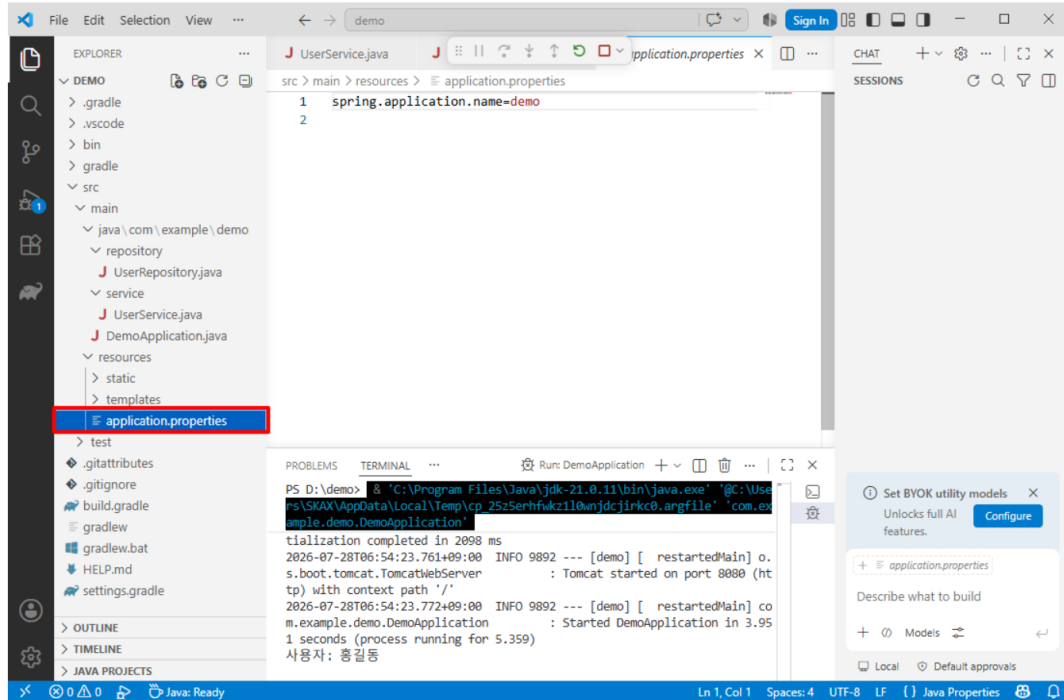
실행 ⇒ UserService의 printUser() 메서드 실행 결과가 출력되는 것을 확인



P99 Configuration과 Profile 관리

설정 파일의 형식을 properties에서 yaml로 변경

src/main/resources/application.properties 파일 이름을 src/main/resources/application.yaml로 변경



설정 파일의 내용을 YAML 형식으로 변경하고, 설정 정보를 추가

사용자 정의 설정 정보

```
app:
  name: My Demo App
  version: 1.0.0
  mode: local
```

서비스 포트

```
server:
  port: 8080
```

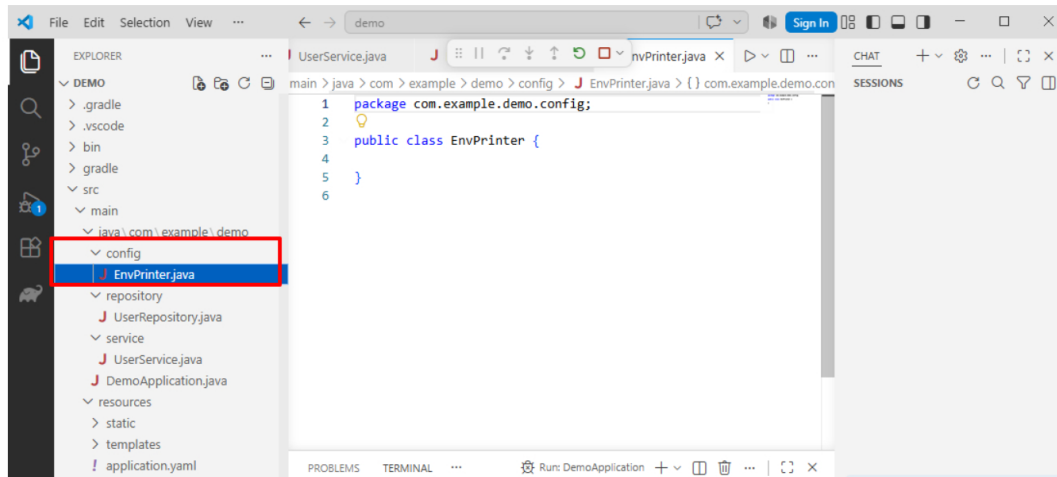
애플리케이션 클래스(DemoApplication) 수정

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

config 패키지 및 EnvPrinter 클래스 추가



```
package com.example.demo.config;
```

```
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.CommandLineRunner;
import org.springframework.core.env.Environment;
import org.springframework.stereotype.Component;
```

// @Component: 스프링이 이 클래스를 빈(Beans)으로 자동 등록
// CommandLineRunner: 스프링 부트 애플리케이션이 완전히 구동된 직후에 run 메서드 내의 코드를 자동으로 실행해 주는 인터페이스

```
@Component
```

```
public class EnvPrinter implements CommandLineRunner {
    private final Environment env;
```

```
    // @Value("${app.name}"): 설정 파일에 있는 app.name 값을 읽어와 appName 변수에 주입
    @Value("${app.name}")
    private String appName;
```

```
    @Value("${app.version}")
    private String appVersion;
```

```
    // 생성자를 이용한 의존 주입
    public EnvPrinter(Environment env) {
```

```
        this.env = env;
    }
```

```
@Override
```

```
public void run(String ...args) throws Exception {
    System.out.println("Spring Boot Environment 실행");
```

```
    // 1. Environment 객체를 통한 값 조회
    System.out.println("server.port = " + env.getProperty("server.port"));
    System.out.println("app.mode = " + env.getProperty("app.mode"));
```

```
    // 2. @Value로 주입받은 값
    System.out.println("app.name = " + appName);
    System.out.println("app.version = " + appVersion);
```

```
    // 3. 활성 프로파일 출력
    String[] activeProfiles = env.getActiveProfiles();
    if (activeProfiles.length == 0) {
        System.out.println("activeProfiles = (none)");
    } else {
        System.out.println("activeProfiles = " + String.join(", ", activeProfiles));
    }
}
```

```
System.out.println("Environment 테스트 완료");
```

```
}
```


실행 ⇒ 터미널에 설정 정보가 출력되는 것을 확인

```
2026-07-28T07:17:58.202+09:00 INFO 19020 --- [ restartedMain] com.example.demo.DemoApplication : Started
DemoApplication in 0.332 seconds (process running for 232.667)
String Boot Environment 실행
server.port = 8080
app.mode = local
app.name = My Demo App
app.version = 1.0.0
activeProfiles = (none)
Environment 테스트 완료
```

P104

설정 파일(application.yaml)에 설정 추가

```
# 사용자 정의 설정 정보
app:
  name: My Demo App
  version: 1.0.0
  mode: local
```

```
# 서비스 포트
server:
  port: 8080

# 애플리케이션 이름
spring:
  application:
    name: demo
```

DemoApplication.java 수정 ⇐ CommandLineRunner 인터페이스를 상속

```
package com.example.demo;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication implements CommandLineRunner {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
        System.out.println("DemoApplication");
    }

    @Value("${spring.application.name}")
    private String appName;

    @Value("${server.port}")
    private int port;

    // 프로퍼티 값 내에서 콜론(:)은 기본값(Default)을 지정할 때 사용
    // spring.application.name이 없을 경우 app을 기본값으로 사용하겠다는 의미
    @Value("${spring.application.name:app}-${server.port:8081}")
    private String composed;

    @Override
    public void run(String... args) throws Exception {
        System.out.println("appName: " + appName);
        System.out.println("port: " + port);
    }
}
```

```
        System.out.println("composed: " + composed);
    }
}
```

실행 ⇒ 실행 순서 확인

```
2026-07-28T07:32:11.819+09:00 INFO 17916 --- [demo] [ restartedMain] com.example.demo.DemoApplication :
Started DemoApplication in 4.016 seconds (process running for 5.464)
```

```
String Boot Environment 실습
server.port = 8080
app.mode = local
app.name = My Demo App
app.version = 1.0.0
activeProfiles = (none)
Environment 테스트 완료
appName: demo
port: 8080
composed: demo-8080
DemoApplication
```

P107 @ConfigurationProperties

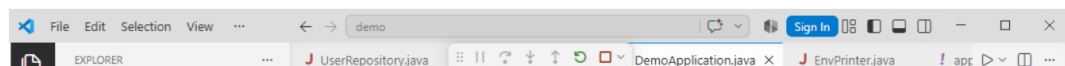
@ConfigurationProperties

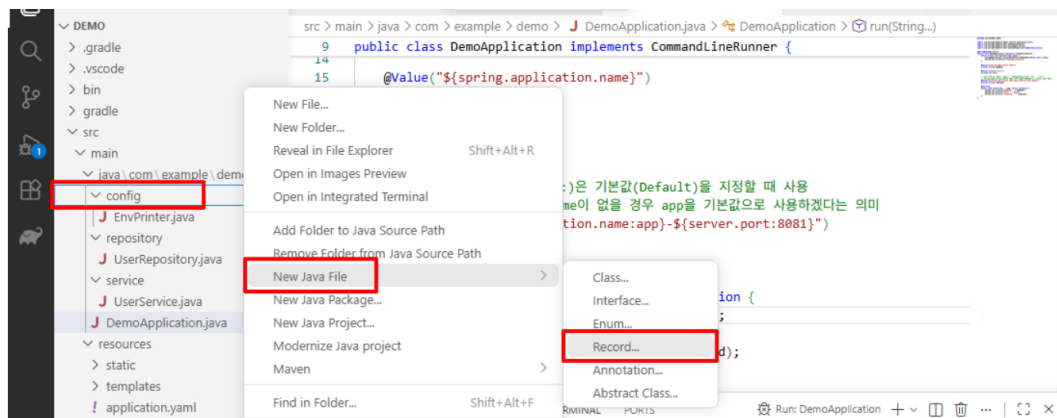
설정 파일(application.yml이나 application.properties)에 정의된 여러 설정값들을 자바 객체(Java Bean)로 타입 안전하게(Type-Safe) 한 번에 바인딩(매핑)해 주는 어노테이션

설정 파일 수정 (들려쓰기 주의)

```
# 사용자 정의 설정
app:
  name: demo
  cors:
    allowed-origins:
      - https://a.com
      - https://b.com
  metadata:
    owner: ops
    region: ap-northeast-2
  admins: ["alice", "bob"]
  timeout: 30s
  max-upload: 25MB
  mode: PRO
```

AppProps 레코드 추가 (config 패키지 아래에)





```
package com.example.demo.config;

import java.time.Duration;
import java.util.List;
import java.util.Map;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.util.unit.DataSize;

@ConfigurationProperties(prefix = "app")
public record AppProps(
    String name,
    Cors cors,
    Map<String, String> metadata,
    List<String> admins,
    Duration timeout,
    DataSize maxUpload,
    Mode mode
) {
    public record Cors(List<String> allowedOrigins) {
    }
    public enum Mode {BASIC, PRO, ENTERPRISE }
}
```

애플리케이션 클래스(DemoApplication) 수정